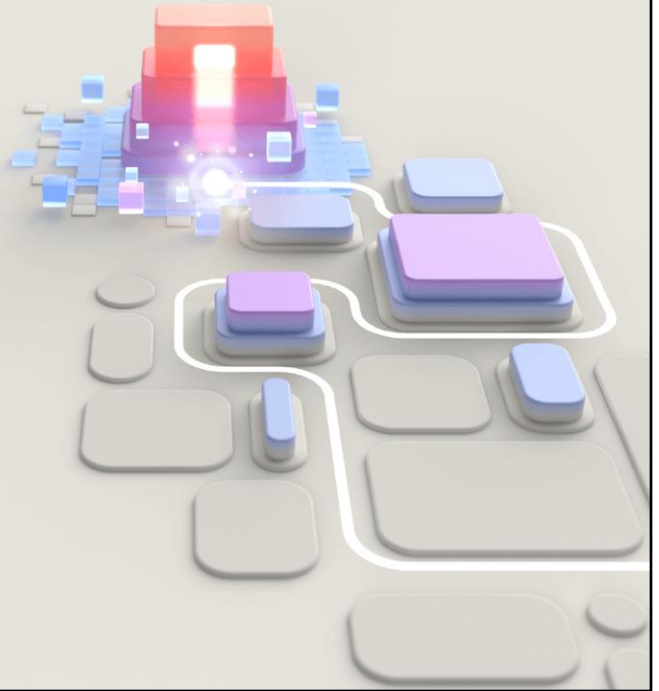




PL-400T00

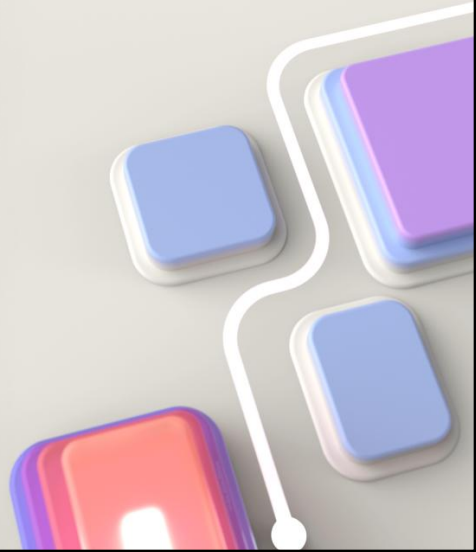
Microsoft Power Platform Developer

© Copyright Microsoft Corporation. All rights reserved.



Integrate Dataverse and Azure

© Copyright Microsoft Corporation. All rights reserved.



Modules



- Integrate with Azure
- Integrate with Dataverse

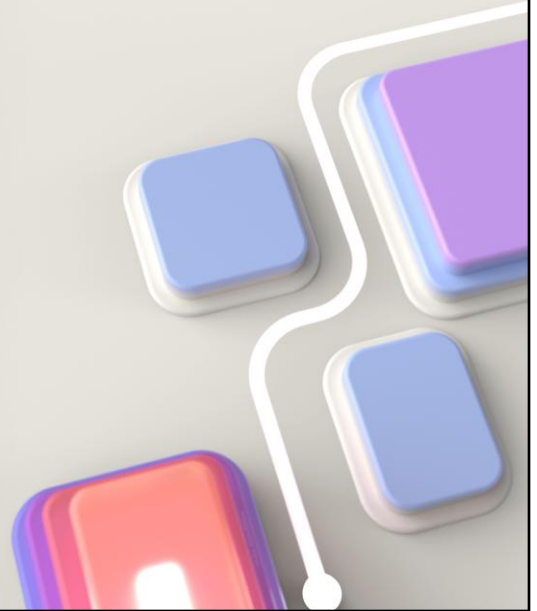
© Copyright Microsoft Corporation. All rights reserved.

Modules

1. Integrate with Azure
2. Integrate with Dataverse

Module 1: Integrate with Azure

© Copyright Microsoft Corporation. All rights reserved.



Learning Objectives

After completing this module, you will be able to:

- 1 Publish Dataverse events to Microsoft Azure Service Bus.
- 2 Write a Service Bus Event Listener that consumes Dataverse events.

© Copyright Microsoft Corporation. All rights reserved.

Microsoft Learn module

Integrate Dataverse Azure solutions

<https://learn.microsoft.com/training/modules/integrate-common-data-service-azure-solutions>



© Copyright Microsoft Corporation. All rights reserved.

Link to student courseware on Learn

Integrate Dataverse Azure solutions

<https://learn.microsoft.com/training/modules/integrate-common-data-service-azure-solutions>

Native Power Platform Azure integrations

Microsoft Entra ID

- Authentication
- Register apps

Azure Service Bus

Azure Event Hub

Logic apps

- Microsoft Dataverse connector

Azure Synapse Analytics

- Azure Synapse link

Azure Bot Service

- Skills to link to copilot bots

Virtual tables

- Azure SQL
- Cosmos DB

Connectors to Azure Services

- Azure SQL
- Azure Storage
- Cognitive Services

© Copyright Microsoft Corporation. All rights reserved.

<https://learn.microsoft.com/en-us/training/modules/integrate-common-data-service-azure-solutions/1-introduction>

<https://learn.microsoft.com/en-us/power-apps/developer/data-platform/azure-integration>

Azure cloud services provide a wealth of functionality and enable Microsoft Power Platform developers to harness its capabilities through various extensibility points. This unit reviews a few Azure technologies in the context of how they play, or potentially play, a role in a Microsoft Power Platform focused implementation. Azure includes many services, and all Microsoft Power Platform developers should be familiar with what Azure has to offer.

Developers should become familiar with the low-code capabilities of Microsoft Power Platform, so they don't rebuild something that it already does well. If you're coming from an Azure developer background, it might be easy to start with an Azure component instead of a Microsoft Power Platform component.

For example, if you were asked to build a chatbot you might first look to Azure Bot Framework. As a Microsoft Power Platform focused developer, you could first look to Power Virtual Agents to implement the chatbot. Then use an Azure Bot Framework Skill to augment the capabilities of the Power Virtual Agent chatbot that you built. Using this approach, solutions are built using the rapid application development of Microsoft Power Platform but still take advantage of the deep customizations that is possible by including Azure services and custom code.

Comparing Azure's integration solutions

Logic Apps – Azure Logic Apps provides a robust visual interface in which you can orchestrate complex integrations across your various environments.

Azure Service Bus – Azure Service Bus is Microsoft's cloud messaging as a service (MaaS) platform.

Azure Functions – Serverless code that can be triggered from Dataverse and other Azure services.

Event Grid – Microsoft Azure Event Grid is an event-driven, publish-subscribe framework that allows you to handle various events.

Event Hubs – Azure Event Hubs provides a real-time data ingestion service that supports millions of events per second.

© Copyright Microsoft Corporation. All rights reserved.

Azure comes with a variety of other solutions, which can also be integrated with Dataverse. The following sections provide a summary of each solution. However, this module will limit further education in future lessons to the solutions that have pre-configured utilities within Dataverse that help streamline your integrations, such as Microsoft Azure Service Bus, Microsoft Azure Event Hubs, and Microsoft Azure Logic Apps.

Logic Apps

Azure Logic Apps provides a robust visual interface in which you can orchestrate complex integrations across your various environments. It has many of the same capabilities as Microsoft Power Automate workflows, so if you are familiar with preparing workflows in that context, you should be able to recognize their similarities. Both provide the ability to use pre-built and custom-built connectors, which you can use to connect to whichever system is needed. Most importantly, Logic Apps comes with a Microsoft Dataverse connector that will allow you to trigger actions based on various Dataverse events (such as a row being created or updated).

Azure Service Bus

Azure Service Bus is Microsoft's cloud messaging as a service (MaaS) platform. Messages are sent to and received from queues for point-to-point communications. Service Bus also provides a publish-subscribe mechanism that uses its Topics feature, which will not be covered in this module.

Azure Functions

Serverless compute
Covered later in this LP

Azure API Management

Because API needs are handled by Dataverse Web API, and because Dataverse provides a role-based security model that can handle most integration scenarios, you might not need to consider Microsoft Azure API Management often. However, if you ever need to write a custom and/or proprietary API to sit on top of the Dataverse Web API, you could use Azure's API Management services.

Event Grid

Microsoft Azure Event Grid is an event-driven, publish-subscribe framework that allows you to handle various events. While Dataverse does not provide out-of-the-box capabilities to integrate with an Event Grid instance as it does with Service Bus or Event Hubs, it is a viable item to consider when you are in need of event-driven integrations.

Event Hubs

Azure Event provides a real-time data ingestion service that supports millions of events per second. This service is good for large streams of data that need to be ingested in real time (which might occur when you are trying to capture items like application telemetry within a business application) but are not common in most business application integration scenarios. Technically, Event Hubs is not seen as an integration solution in Azure (it's seen as an analytics solution given that its predominant applications are with "big data"). Dataverse provides a utility to publish events to an Event Hub. In the context of Dataverse, while provided as an endpoint, Azure Service Bus traditionally becomes the preferred method for integration of actual business data because it comes with far less overhead.

Dataverse outbound integration with Azure

Azure Service Bus

- No code and code integration
- Post messages to Service Bus Queues, Topics, Relays
- Post Operation only

Azure Event Hub

- No code and code integration
- Post messages and events to Event Hub
- Post Operation only

Webhook

- Code and no code integration
- Simple HTTP trigger

© Copyright Microsoft Corporation. All rights reserved.

<https://learn.microsoft.com/power-apps/developer/data-platform/azure-integration>

Execution context posted to the Azure service

Choosing the right Azure integration solution

IF YOU WANT TO...

- Perform workflow and orchestrate within and across systems at scale and with monitoring and error handling
- Connect on-premises and cloud systems or decouple transactional systems
- Process streaming data
- Publish and subscribe to events
- Send event data to external HTTP endpoints
- Allow external systems to read and write Dataverse data

USE THIS

- Logic Apps
- Service Bus
- Event Hubs
- Event Grid
- Webhook
- Custom API

© Copyright Microsoft Corporation. All rights reserved.

If you're struggling to figure out which Azure integration solution best suits your needs, consider the information in the following table.

For in-depth guidance on Azure's broader Integration Services framework, refer to the Azure Integration Services whitepaper, which is found [here](#).

Another article to reference is one found on the Azure website

Choose between Azure messaging services - Event Grid, Event Hubs, and Service Bus

<https://learn.microsoft.com/azure/service-bus-messaging/compare-messaging-services>

<https://learn.microsoft.com/power-apps/developer/data-platform/use-webhooks>

Integration with Azure Service Bus Queue

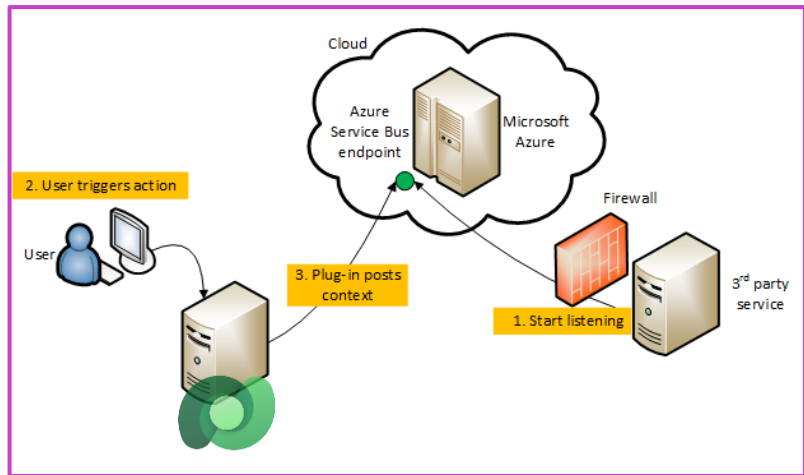
Generate Shared Access Signature (SAS) token

Send execution context as

- Binary
- JSON
- XML

Queue Listener

- Logic app
- Azure Function
- Custom code



© Copyright Microsoft Corporation. All rights reserved.

<https://learn.microsoft.com/power-apps/developer/data-platform/azure-integration>

Demo: Azure Service Bus

DEMO

© Copyright Microsoft Corporation. All rights reserved.

<https://learn.microsoft.com/power-apps/developer/data-platform/walkthrough-configure-azure-sas-integration>
<https://learn.microsoft.com/power-apps/developer/data-platform/azure-integration>

Create Azure Service Bus and Queue
Register Service Endpoint
Register Step on Create of table
Use Azure Portal to show message on Queue

Publish event using the IServiceEndpointNotificationService

- Create a plug-in
- Requires the ID of the service endpoint
- Use IServiceEndpointNotificationService interface
- Execute using the serviceendpoint table
- Pass your custom message as second parameter

© Copyright Microsoft Corporation. All rights reserved.

```
public void Execute(IServiceProvider serviceProvider)
{
    IServiceEndpointNotificationService
    cloudService =
    (IServiceEndpointNotificationService)ser
    viceProvider.GetService(typeof(IServiceE
    ndpointNotificationService));

    var response = cloudService.Execute(new
    EntityReference("serviceendpoint",
    serviceEndpointId), context);
}
```

<https://learn.microsoft.com/power-apps/developer/data-platform/write-custom-azure-aware-plugin>

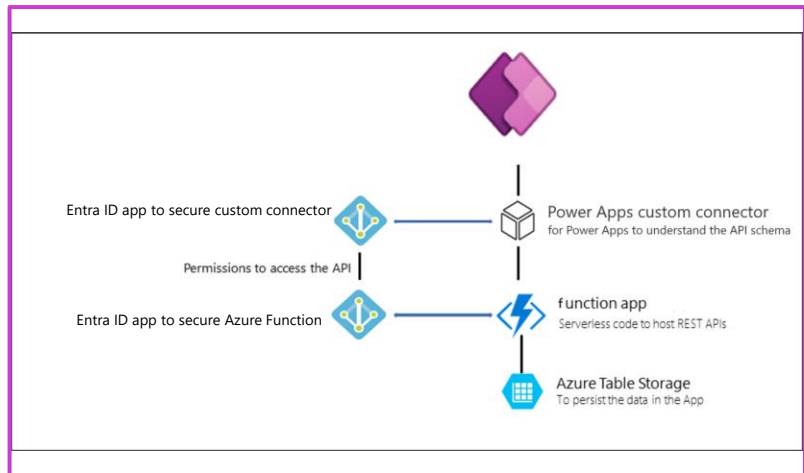
Azure functions

HTTP endpoints

Build APIs and control
with Azure API
Management

Create custom connector
for use with flows and
canvas apps

Offload work from
Dataverse

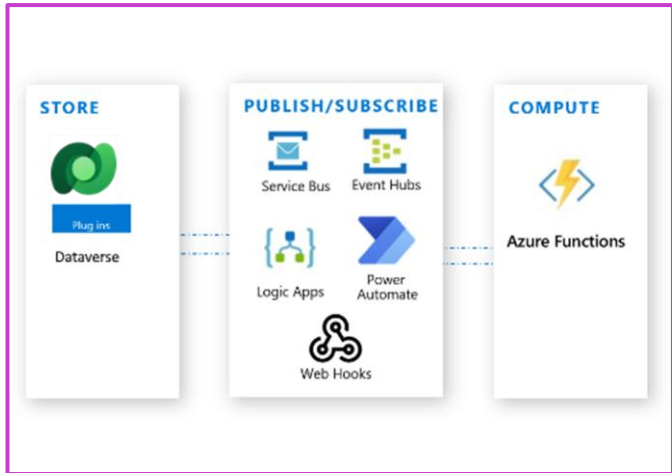


© Copyright Microsoft Corporation. All rights reserved.

[Azure Functions](#) is a supported extensibility endpoint for Power Platform. Extracting custom logic into an Azure function enables you to offload complex logic outside of your transactional application, thus providing a much more stable and effective user experience. Azure Functions can also be used to build APIs that are exposed to Microsoft Power Platform as a custom connector. As a custom connector the Azure Function business logic can easily be invoked from apps and flows.

Discover Azure Functions

- Azure Functions are a great solution for processing data, integrating systems, working with the internet-of-things (IoT), and building simple APIs and microservices.
- Consider Functions for tasks like image or order processing, file maintenance, or for any tasks that you want to run on a schedule.
- Azure Functions supports *triggers*, which are ways to start execution of your code, and *bindings*, which are ways to simplify coding for input and output data.
- Azure Function with HTTP trigger can be called using a Webhook



© Copyright Microsoft Corporation. All rights reserved.

Dataverse and Power Platform can also trigger Azure Functions

Serverless code

Build using

- Azure Portal
- Visual Studio
- Visual Studio Code

<https://learn.microsoft.com/training/modules/integrate-common-data-service-azure-solutions/5-azure-function>

Compare Azure Functions hosting plans

The following is a summary of the benefits of the three main hosting plans for Functions:

Consumption plan

- The default hosting plan.
- Scales automatically and you only pay when your functions are running.
- Instances of the Functions host are dynamically added and removed based on the number of incoming events.

Premium plan

- Automatically scales based on demand using pre-warmed workers.
- Runs on more powerful instances and connects to virtual networks.

Dedicated plan

- Run your functions within an App Service plan at regular App Service plan rates.
- Best for long-running scenarios where Durable Functions can't be used.

© Copyright Microsoft Corporation. All rights reserved.

Compare Azure Functions hosting plans

Function app timeout duration

- The `functionTimeout` property in the `host.json` project file specifies the timeout duration.
- **Consumption** plan has a default timeout of 5 minutes, and a maximum timeout of 10 minutes.
- **Premium** and **Dedicated** plans have a default timeout of 302 minutes and no maximum duration.

Storage account requirements

- On any plan, a function app requires a general Azure Storage account.
- Azure Functions relies on Azure Storage for operations such as managing triggers and logging function executions.

© Copyright Microsoft Corporation. All rights reserved.

Storage account requirements

These accounts, which include blob-only storage accounts (including premium storage) and general-purpose storage accounts with zone-redundant storage replication, are filtered-out from your existing Storage Account selections when you create a function app.

The same storage account used by your function app can also be used by your triggers and bindings to store your application data. However, for storage-intensive operations, you should use a separate storage account.

Azure Functions development

A function contains two important pieces:

- **Your code**, which can be written in a variety of languages
- **Some config**, the *function.json* file.

The *function.json* file defines the function's trigger, bindings, and other configuration settings. Every function has one and only one trigger. The runtime uses this config file to determine the events to monitor and how to pass data into and return data from a function execution.

© Copyright Microsoft Corporation. All rights reserved.

```
{
  "disabled": false,
  "bindings": [
    // ... bindings here
    {
      "type": "bindingType",
      "direction": "in",
      "name": "myParamName",
      // ... more depending on
binding
    }
  ]
}
```

The bindings property is where you configure both triggers and bindings. Each binding shares a few common settings and some settings which are specific to a particular type of binding. Every binding requires the following settings:

- **type** / string - Name of binding. For example, queueTrigger.
- **direction** / string - Indicates whether the binding is for receiving data into the function or sending data from the function. For example, in or out.
- **name** / string - The name that is used for the bound data in the function. For example, myQueue.

Azure Functions triggers and bindings

- Triggers are what cause a function to run. A trigger defines how a function is invoked and a function must have exactly one trigger.
- Binding to a function is a way of declaratively connecting another resource to the function; bindings may be connected as input bindings, output bindings, or both.
- You can mix and match different bindings to suit your needs.
- Triggers and bindings let you avoid hardcoding access to other services.

© Copyright Microsoft Corporation. All rights reserved.

Triggers and bindings let you avoid hardcoding access to other services:

- Your function receives data (for example, the content of a queue message) in function parameters.
- You send data (for example, to create a queue message) by using the return value of the function.

Azure Functions vs. Dataverse Plug-ins

Azure Functions

- Consumption plan 5-minute default and can be increased to 10-minute
- Durable functions allow chaining of functions
- Premium and Dedicated plans have longer
- Need to Authenticate

Dataverse Plug-in

- Event driven
- Cannot schedule
- 2-minute limit
- Authenticated automatically

© Copyright Microsoft Corporation. All rights reserved.

Microsoft Azure Functions provides a great mechanism for doing small units of work, similar to what you would use plug-ins for in Dataverse. In many scenarios it might make sense to offload this logic into a separate component, such as an Azure Function, to reduce load on the Dataverse's application host. You have the availability to run functions in a synchronous capacity because Dataverse webhooks provide the Remote Execution Context of the given request.

However, Azure Functions does not explicitly run within the Dataverse's implementation pipeline, so if you need to update data in the most high-performing manner, such as autoformatting a string value before it posts to Dataverse, we still recommend that you use a plug-in to perform this type of operation.

Publish Dataverse events using webhooks

Another method for publishing events from Dataverse to an external service is to register a webhook

- Name
- Endpoint URL
- Authentication

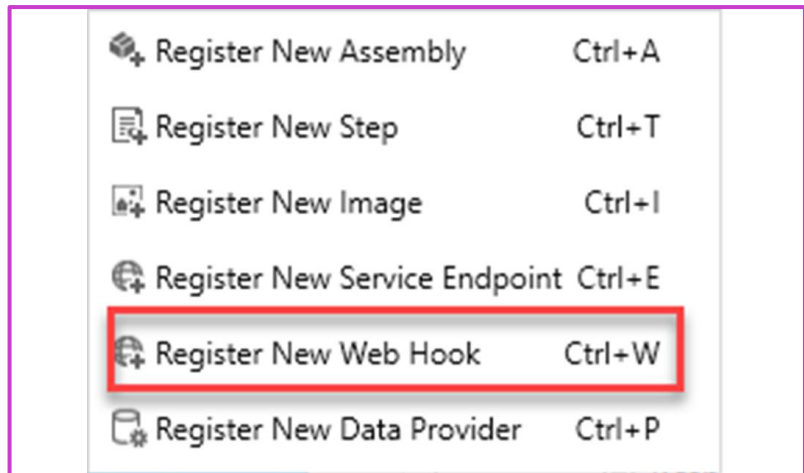
© Copyright Microsoft Corporation. All rights reserved.

Another method for publishing events from Dataverse to an external service is to register webhooks. A webhook is an HTTP-based mechanism for publishing events to any Web API-based service of your choosing. This method allows you to write your own custom code that is hosted on external services as a point-to-point.

<https://learn.microsoft.com/power-apps/developer/data-platform/use-webhooks>

Register a Webhook endpoint

Webhook endpoint registration is performed similarly to Service Endpoint registration, by using the Plug-in Registration Tool.



© Copyright Microsoft Corporation. All rights reserved.

Webhook endpoint registration is performed similarly to Service Endpoint registration, by using the Plug-in Registration Tool.

Within the Plug-in Registration Tool, you can register a new webhook by selecting **Register New Web Hook** under the **Register** menu option.

The following **WebHook Registration** dialog box will appear, where you can configure the URL of your endpoint, along with any authentication options.

Webhook authentication options

The following table describes the three authentication options that you can use to consume a webhook message from a given endpoint.

Type	Description
HttpHeader	Includes one or more key value pairs in the header of the HTTP request. Example: Key1: Value1, Key2: Value2
WebhookKey	Includes a query string by using code as the key and a value that is required by the endpoint. When registering the webhook by using the Plug-in Registration Tool, only enter the value. Example: ?code=00000000-0000-0000-0000-000000000001
HttpQueryString	Includes one or more key value pairs as query string parameters. Example: ?Key1=Value1&Key2=Value2

© Copyright Microsoft Corporation. All rights reserved.

The following table describes the three authentication options that you can use to consume a webhook message from a given endpoint.

Type Description

HttpHeader

Includes one or more key value pairs in the header of the HTTP request. Example: Key1: Value1, Key2: Value2

WebhookKey

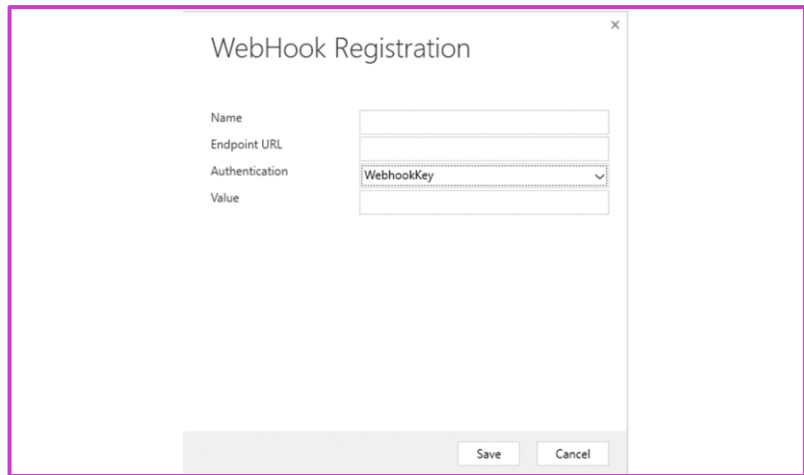
Includes a query string by using code as the key and a value that is required by the endpoint. When registering the webhook by using the Plug-in Registration Tool, only enter the value. Example: ?code=00000000-0000-0000-0000-000000000001

HttpQueryString

Includes one or more key value pairs as query string parameters. Example: ?Key1=Value1&Key2=Value2

Register with WebhookKey Authentication

If **WebhookKey** is specified as the **Authentication** method, a query string is passed to the URL with the given key in the format `?code=[web hook key]`.

A screenshot of a 'WebHook Registration' dialog box. The dialog has a title bar with a close button (X). It contains four labels on the left: 'Name', 'Endpoint URL', 'Authentication', and 'Value'. To the right of these labels are input fields. The 'Authentication' field is a dropdown menu with 'WebhookKey' selected. At the bottom right of the dialog are 'Save' and 'Cancel' buttons.

Field	Value
Name	
Endpoint URL	
Authentication	WebhookKey
Value	

© Copyright Microsoft Corporation. All rights reserved.

If **WebhookKey** is specified as the **Authentication** method, a query string is passed to the URL with the given key in the format `?code=[web hook key]`. This method is handy when you are calling Azure Functions because it uses this code parameter by default to perform its authentication.

Webhooks vs. Azure Service Bus

Service Bus

- Asynchronous
- Register with Plugin Registration Tool
- Invoke with no code or with code
- Requests can be queued

Webhook

- Synchronous or Asynchronous
- Register with Plugin Registration Tool
- Invoke with no code or with code
- Requests dependent on HTTP endpoint being available

© Copyright Microsoft Corporation. All rights reserved.

When considering integration mechanisms, you have a few available options. It's important that you consider various elements when choosing a given method.

Consider using Azure Service Bus when:

- High scale asynchronous processing/queueing is a requirement.
- Multiple subscribers might be needed to consume a given Dataverse event.
- You want to govern your integration architecture in a centralized location.

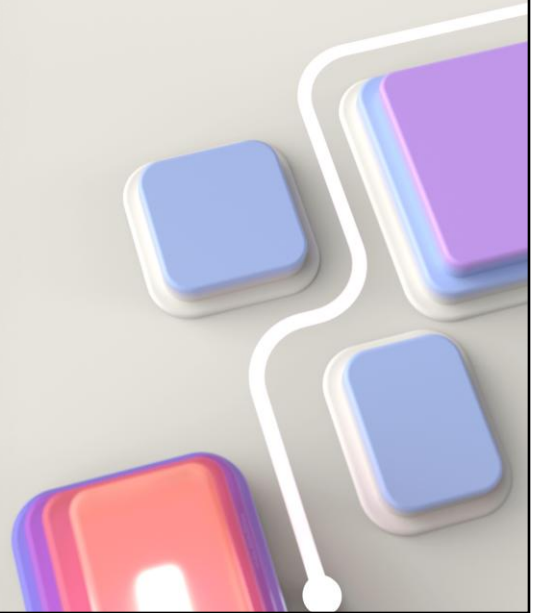
Consider using webhooks when:

- Synchronous processing against an external system is required as part of your process (Dataverse only supports asynchronous processing against Service Bus Endpoints).
- The external operation that you are performing needs to occur immediately.
- You want the entire transaction to fail unless the webhook payload is successfully processed by the external service.
- A third-party Web API endpoint already exists that you want to use for integration purposes.
- SAS authentication isn't preferred and/or feasible (webhooks support authentication through authentication headers and query string parameter keys).

<https://learn.microsoft.com/power-apps/developer/data-platform/use-webhooks>

Module 2: Integrate with Dataverse

© Copyright Microsoft Corporation. All rights reserved.



Change Tracking
Alternate Keys
Upsert

Learning Objectives

After completing this module, you will be able to:

- 1 Work with change tracking
- 2 Alternate keys
- 3 Use upsert method

© Copyright Microsoft Corporation. All rights reserved.

Integrate with Dataverse

There are several features of Dataverse that external systems can use to integrate with Dataverse:

- Change tracking
- Alternate keys
- Upsert method

© Copyright Microsoft Corporation. All rights reserved.

Change Tracking

- Pulls changed records from Dataverse
- Enable on table metadata
- Note: Change Tracking is not associated with Auditing
- Retrieve list of changes since last request using a token
- Set Token to null for initial extract
- Store token and use to call to next time
- Multiple Tokens allowed
- Tokens expire after 30 days by default

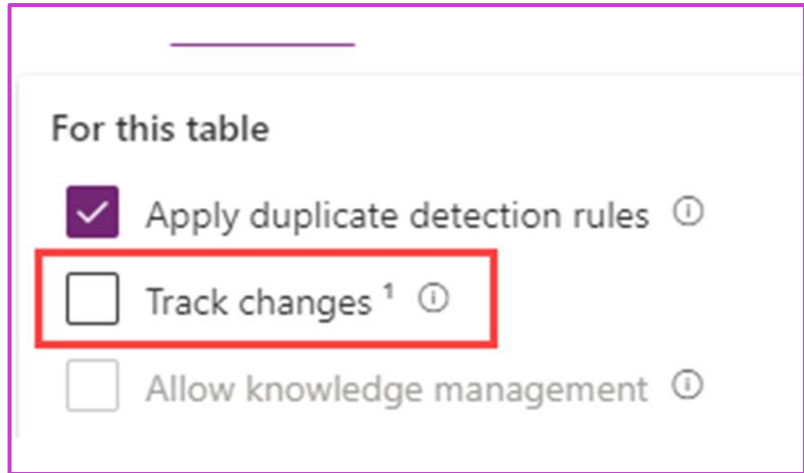
© Copyright Microsoft Corporation. All rights reserved.

Configure table change tracking
Enable change tracking to control data
synchronization

<https://learn.microsoft.com/power-apps/developer/data-platform/use-change-tracking-synchronize-data-external-systems>

Enable change tracking for a table

You can enable change tracking on a table by setting the **Track changes** flag on the table properties in the Power Apps maker portal



© Copyright Microsoft Corporation. All rights reserved.

Track table data changes with change tracking and the Web API

Commonly, organizations need to synchronize data across multiple systems. Microsoft Dataverse provides a feature to efficiently manage this scenario, called *change tracking*. With change tracking enabled, you can build applications to query for these changes.

You can enable change tracking on an table by setting the **Track Changes** flag on the table settings in the PowerApps maker portal, found under the **Advanced Settings > For this table** section.

Read table change records by using platform APIs

Organization Service

RetrieveEntityChanges message

Loop through EntityChanges.Changes

Change types

- NewOrUpdatedItem
- RemoveOrDeleted

Web API

Specify odata.track-changes preference in the request

Add deltatoken to URL parameters

Returns deltaLink

© Copyright Microsoft Corporation. All rights reserved.

According to the OData 4.0 documentation, delta links are "opaque, service-generated links that the client uses to retrieve subsequent changes to a result." Microsoft has built this standard into change tracking in Dataverse to provide a common, standard method for interfacing with change data as it occurs over time.

After issuing a change tracking request, your response will contain a delta link, which you'll then pass to your next request to get changes that have occurred since the initial request. It will be up to your app to maintain the delta link, similarly to how you might have done tracking with date/time stamps in the past.

<https://learn.microsoft.com/power-apps/developer/data-platform/use-change-tracking-synchronize-data-external-systems>

Retrieve changes in tables by using the Web API

OData

```
GET [Organization
URI]/org1/api/data/v9.1/accounts?$select=
name,accountnumber,telephone1,fax
HTTP/1.1
Prefer: odata.track-changes
Cache-Control: no-cache
OData-Version: 4.0
Content-Type: application/json
```

OData

```
{
  "@odata.context":"[Organization
URI]/data/v9.0/$metadata#accounts(name,telephone1,fax)/$delta",
  "@odata.deltaLink":"[Organization
URI]/api/data/v9.1/accounts?$select=name,telephone1,fax&$deltatoken=919058%2108%2f22%2f2017%2008
%3a21%3a20",
  "value":
  [
    {
      "@odata.etag":"W/\"915244\"",
      "name":"Monte Orton",
      "telephone1":"555000",
      "fax":"10101",
      "accountid":"60c4e274-0d87-e711-80e5-00155db19e6d"
    },
    {
      "@odata.context":"[Organization
URI]/api/data/v9.0/$metadata#accounts/$deletedEntity",
      "id":"2e451703-c686-e711-80e5-00155db19e6d",
      "reason":"deleted"
    }
  ]
}
```

© Copyright Microsoft Corporation. All rights reserved.

After you've enabled change tracking on a table, you can retrieve its changes through the Web API by adding `odata.track-changes` as a preference header in your web request. The following example requests changes from the account table for a specific set of columns (name, account number, telephone1, and fax).

On the initial request, you'll receive the entire set of data that applies to the query because a delta has not been applied. However, the response will show an `@odata.deltaLink` value. This link contains a `deltatoken` parameter, which is intended to be passed into your subsequent request when you want to see what changes have been applied since this query was run. Think of this process as similar to how you would handle paging in OData.

The following example is a response to the preceding account request:

In this scenario, consider that some time has passed, and you now want to see if any other changes have been applied. You can use the `@odata.deltaLink` parameter to run your query (or extract its `deltatoken` if you are managing connection strings differently), and see what changes have happened since your last request.

Retrieve changes in tables by using the Web API

OData

GET [Organization URI]/api/data/v9.1/accounts?\$select=name,accountnumber,telephone1,fax&\$deltatoken=919042%2108%2f22%2f2017%2008%3a10%3a44

```
{
  "@odata.context":"[Organization URI]/data/v9.1/$metadata#accounts(name,telephone1,fax)/$delta",
  "@odata.deltaLink":"[Organization URI]/api/data/v9.1/accounts?$select=name,telephone1,fax&$deltatoken=919058%2108%2f22%2f2017%2008%3a21%3a20",
  "value":
  [
    {
      "@odata.etag":"W/\\"915244\\"",
      "name":"Monte Orton",
      "telephone1":"555000",
      "fax":"10101",
      "accountid":"60c4e274-0d87-e711-80e5-00155db19e6d"
    },
    {
      "@odata.context":"[Organization URI]/api/data/v9.1/$metadata#accounts/$deletedEntity",
      "id":"2e451703-c686-e711-80e5-00155db19e6d",
      "reason":"deleted"
    }
  ]
}
```

© Copyright Microsoft Corporation. All rights reserved.

The following example shows how you would retrieve changes since that last request by running the returned `@odata.deltaLink`:

In this scenario, imagine that a new account was created and another one was deleted. Your response would then look similar to the following example:

As shown in the example, another delta link was provided, which you can then use the next time you need to see what changes have occurred.

Alternate keys

- For inbound integration into Dataverse
- Create for existing column(s)
- Indexed
- Performance

```
using (var service = new OrganizationService(crmConnection))
{
    // Use alternate key (accountnumber) column to identify an account row
    Entity account = new Entity("account", "accountnumber", "MIC-12345");

    // Set new credit limit;
    account["creditlimit"] = new Money(100000);

    UpdateRequest request = new UpdateRequest() { Target = account };
    UpdateResponse response = (UpdateResponse)service.Execute(request);
}
```

© Copyright Microsoft Corporation. All rights reserved.

For pushing updated records into Dataverse records
Specify column(s) for alternate key on table
Creates unique nullable index on the column(s)
In Organization service, must use UpdateRequest

Upsert method

- Insert or Update in one request
- Use when don't know if row exists
- Need to provide "enough" information

```
using (var service = new OrganizationService(crmConnection))
{
    // Use alternate key (accountnumber) column to identify an account row
    var account = new Entity("account")
    {
        KeyAttributes = new KeyAttributeCollection
        {
            {"accountnumber", "MIC-12345" }
        }
    };

    account["name"] = "Microsoft";
    account["creditlimit"] = new Money(100000);

    var request = new UpsertRequest() { Target = account };
    var response = (UpsertResponse)service.Execute(request);
}
```

© Copyright Microsoft Corporation. All rights reserved.

For pushing new or
updated records
into Dataverse
records
Works with alternate

keys

<https://learn.microsoft.com/power-apps/developer/data-platform/use-upsert-insert-update-record>

Summary



- Microsoft Dataverse has a wealth of pre-built methods for interacting with Azure solutions
- Azure can be used to distribute workloads away from your Dataverse environment
- Dataverse provides many ways to integrate with external systems

© Copyright Microsoft Corporation. All rights reserved.

Modules

1. Integrate with Azure
2. Integrate with Dataverse



© Copyright Microsoft Corporation. All rights reserved.

1. Integrate with Azure
2. Integrate with Dataverse